

Backtester — Project Guide

A reference for how this project is structured, what each class does, and how everything connects. Read this whenever you've lost track of the big picture or aren't sure where a new piece of logic should live.

1. Core Philosophy

Two rules drive every structural decision in this project:

1. **The engine knows nothing about how it's used.** `engine/` has no `print()` statements, no CLI parsing, no HTTP. It takes structured input (a config) and returns structured output (a result object). The CLI, the future API, and the future frontend are all just different ways of calling the same engine.
2. **Vectorized, not event-driven.** There is no `for bar in bars:` loop anywhere. Strategies compute a signal across an entire price series at once using pandas, and returns/equity are derived with array operations. This is fast and well-suited to research/comparison — the tradeoff is it's not realistic enough for live trading (see § 7 if that changes later).

If you keep those two things true, the project stays easy to extend.

2. Project Structure

```

financeBacktester/
├── engine/
│   ├── data/
│   │   ├── models.py          # Bar – validates a single OHLCV row
│   │   └── loader.py          # DataLoader – fetches & cleans price data
│   ├── strategy/
│   │   ├── base.py            # Strategy – abstract interface
│   │   └── examples/
│   │       └── sma_crossover.py
│   ├── backtest.py            # Backtest – the orchestrator / "main
loop"
│   ├── results.py             # BacktestResult – structured output
│   ├── metrics.py             # MetricsCalculator – performance
stats
│   └── config.py              # BacktestConfig – structured
input
├── cli/
│   └── run_backtest.py        # thin script: build config →
run → print
├── storage/
│   ├── models.py              # DB schema for saved runs
│   └── db.py                  # save/query functions
├── api/
│   ├── schemas.py             # Pydantic mirrors of
engine types
│   ├── routes/backtests.py    # POST/GET endpoints
│   └── main.py                # FastAPI app
├── frontend/                  # added last
├── tests/
└── requirements.txt

```

3. The Classes

Bar — `engine/data/models.py`

What it is: An immutable, validated representation of a single OHLCV price bar (one symbol, one timestamp, one row of data).

What it does:

- Validates on construction — raises `ValueError` if `high < low`, if `open / close` fall outside the `[low, high]` range, or if `volume` is negative.
- Provides a couple of convenience properties (`typical_price` , `is_bullish`) that strategies or reporting code can reuse instead of reimplementing the same formula everywhere.

How it connects: `Bar` is used **only at the data-loading boundary**.

`DataLoader` constructs a `Bar` from every row it loads, purely to trigger validation — if construction succeeds, the row is trustworthy. After that, `Bar` objects are discarded; the `DataFrame` is what flows through the rest of the engine. Think of `Bar` as a gatekeeper, not a data structure you pass around.

`DataLoader` — `engine/data/loader.py`

What it is: The single place responsible for turning raw price data (from `yfinance` or a CSV) into the engine's standard `DataFrame` shape.

What it does:

- `load(symbol, start, end, source="yfinance")` — the only public method. Fetches raw data, normalizes column names, flattens `yfinance`'s `MultiIndex` columns if present, sets a clean `DatetimeIndex` , sorts and deduplicates, and validates every row through `Bar` .
- Returns a `DataFrame` shaped exactly like this:

```
           open    high    low  close  volume
timestamp
2023-01-03    130.28  130.90  124.17  125.07  112117500
2023-01-04    126.89  128.66  125.08  126.36   89113600
```

How it connects: `Backtest` calls `DataLoader.load()` twice per run — once for the strategy's symbol, once for the benchmark symbol. Nothing else in the engine talks to `yfinance` or reads CSVs directly; if you ever swap data providers, this is the only file that changes.

Strategy (abstract) — `engine/strategy/base.py`

What it is: The interface every trading strategy implements.

What it does:

- Defines one required method: `generate_signals(df: pd.DataFrame) -> pd.Series`.
- Takes the full price DataFrame, returns a `Series` (same index) of position values: `1` = long, `0` = flat, `-1` = short.
- Should be **side-effect free** — it reads `df`, it does not mutate it.
`Backtest` is responsible for assembling the final DataFrame with signals and returns attached.

How it connects: `Backtest` holds one `Strategy` instance and calls `generate_signals()` once per run. Every concrete strategy (e.g. `SMAStrategy` in `engine/strategy/examples/sma_crossover.py`) subclasses this and only needs to implement that one method — everything about execution, returns, and equity calculation is handled elsewhere, so strategy code stays purely about *signal logic*.

Backtest — `engine/backtest.py`

What it is: The orchestrator. This is the closest thing to a "main loop" in the project, even though it's implemented as a sequence of vectorized DataFrame operations rather than a literal loop.

What it does, step by step, inside `run()` :

1. Loads the strategy's symbol data and the benchmark's symbol data via `DataLoader`.
2. Calls `strategy.generate_signals(df)` to get the `signal` column.

3. Computes raw returns (`close.pct_change()`) and strategy returns (`returns * signal.shift(1)` — the `.shift(1)` is what prevents look-ahead bias: today's signal can only affect *tomorrow's* return).
4. Subtracts commission cost on days the position changes.
5. Computes cumulative equity curves (normalized to start at `1.0`) for both the strategy and the benchmark.
6. Aligns both curves on shared dates.
7. Hands the returns series to `MetricsCalculator` to compute performance stats.
8. Packages everything into a `BacktestResult` and returns it.

How it connects: `Backtest` is the hub — it's the only class that talks to `DataLoader` , `Strategy` , and `MetricsCalculator` all at once. It's constructed either directly (in `cli/run_backtest.py`) or from a `BacktestConfig` (once that layer exists), and its `run()` method is the one call the CLI, tests, and eventually the API all make.

BacktestResult — engine/results.py

What it is: A plain data container for everything a completed backtest produced.

What it does: Holds `equity_curve` , `benchmark_curve` , `signals` , and `metrics` (a dict of Sharpe, CAGR, max drawdown, alpha/beta, etc.). No methods, no logic — just structured output.

How it connects: This is what `Backtest.run()` returns, what `cli/run_backtest.py` prints, what `storage/db.py` will persist, and what the future API will serialize to JSON for the frontend chart. Keeping it a dumb container (rather than putting formatting logic on it) means every consumer can shape the data however it needs without fighting the class.

MetricsCalculator — engine/metrics.py

What it is: A stateless collection of performance-statistic functions.

What it does: Takes return series (strategy and/or benchmark) and computes standard metrics: total return, CAGR, Sharpe ratio, max drawdown, alpha/beta vs. the benchmark. Likely implemented as static/class methods since there's no reason for it to hold state.

How it connects: Called once by `Backtest.run()` near the end of the pipeline, after returns have been computed. Its output becomes the `metrics` dict on `BacktestResult`.

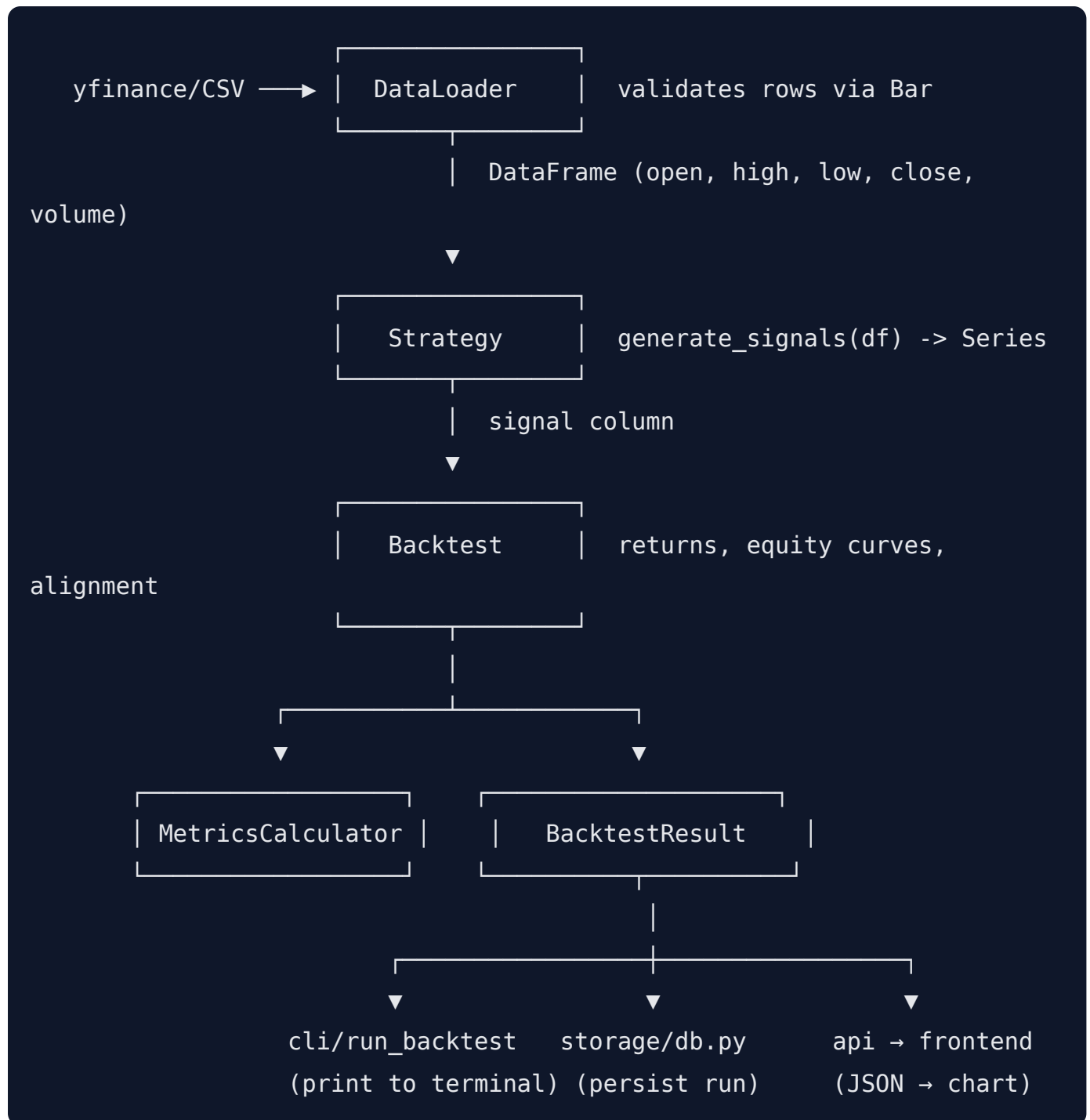
BacktestConfig — engine/config.py (Stage 6 — not built yet)

What it is: A serializable description of everything needed to run one backtest: strategy name, strategy params, symbol, benchmark symbol, date range, initial capital, commission rate.

What it does: Nothing but hold data (a `dataclass` or Pydantic model). Its entire purpose is to be the **single object that fully describes a run**, so a run can be triggered the same way whether it comes from a CLI flag, a saved JSON file, or an HTTP request body.

How it connects: This is what turns `Backtest` into something constructible from an API request later, with zero translation work — the API's request schema will just mirror this class field-for-field.

4. Data Flow (End to End)



Everything above the `BacktestResult` line is the engine. Everything below it is a consumer — and none of the consumers change how the engine works, they just read its output differently.

5. Build Order Recap

Stage	Files	Status
-------	-------	--------

1	<code>engine/data/models.py</code> , <code>engine/data/loader.py</code>	✓ done
2	<code>engine/strategy/base.py</code> , <code>engine/strategy/examples/sma_crossover.py</code>	next
3	<code>engine/backtest.py</code> , <code>engine/results.py</code>	—
4	<code>engine/metrics.py</code>	—
5	<code>cli/run_backtest.py</code> + a throwaway matplotlib plot to sanity-check	—
6	<code>engine/config.py</code>	—
7	<code>storage/models.py</code> , <code>storage/db.py</code>	—
8	<code>api/schemas.py</code> , <code>api/routes/backtests.py</code> , <code>api/main.py</code>	—
9	<code>frontend/</code>	—

Critical path to "see a graph of strategy vs. index": Stages 1 → 2 → 3 → 5.

Everything after that is about making the working engine reusable and servable, not about making it work in the first place.

6. Rules of Thumb While Implementing

- If you're about to add a `print()` or an HTTP call inside `engine/` , **stop**. That belongs in `cli/` or `api/` .
- If a `Strategy` subclass needs more than `generate_signals(df)` , **reconsider**. Params (like MA window lengths) belong in `__init__` , not as extra abstract methods — keep the interface to one method.
- **Every DataFrame flowing through the engine should have a `DatetimeIndex` named `timestamp` and lowercase OHLCV columns.** If a new piece of code produces a DataFrame that doesn't match this shape, fix it at the source rather than patching around it downstream.
- **`.shift(1)` is not optional.** Any time you multiply a signal by a return to compute strategy performance, the signal must be shifted by one bar. Forgetting this is the single most common backtesting bug (look-ahead bias) and it will make every strategy look better than it actually is.

7. If You Ever Need Event-Driven Mode

If a future goal requires realistic order execution or live trading, you'd add a **parallel** path rather than replacing this one: `Broker` , `Portfolio` , `Order` , and `Trade` classes, plus an event-driven `Backtest.run_event_driven()` (or a separate `EventDrivenBacktest` class) that loops bar-by-bar. `Bar` already exists and would become the primary unit passed around again, rather than just a validation gate. This isn't needed for the current goal (strategy vs. index comparison) — noting it here so future-you knows where it would slot in without a rewrite.